

Constella: Teacher Selection and Search Cost in Query-Side Distillation

Dylan Couzon Evgeniya Sukhodolskaya
dylan.couzon@qdrant.com jenny@qdrant.com
Kumar Shivendu Andrey Vasnetsov
kumar.shivendu@qdrant.com andrey@qdrant.com
Qdrant

October 2026

Abstract

A dense document index is expensive to build and slow to change, while the query encoder that searches it can be replaced at will. Query-side distillation exploits this asymmetry: a small student learns to emit queries in the space of a large, frozen document encoder, so the stored vectors stay in place. We build Constella, three compatible query encoders for Stella, a pretrained 400-million-parameter embedding model: Stella’s own query path, Nano, a 34.5-million-parameter transformer, and Zero, a token lookup table that runs no transformer at all. Over 15 BEIR datasets they retain 100%, 90.5%, and 81.4% of exact-search quality at median query-encoding times of 31.6, 2.25, and 0.044 ms on a laptop CPU.

Because the cheap students are fit in closed form in minutes, teacher selection becomes testable. Across 26 public embedding models, a teacher’s own retrieval score was a poor guide to its student’s quality under either of two student recipes. A cheap development screen of the student itself tracked the public ranking, and the strongest teacher produced one of the weakest students. The savings also move. Over each teacher’s unchanged index, table queries recover fewer of their exact neighbors than the teacher’s queries at equal graph effort, in all 25 measured spaces on two corpora. Matching the teacher takes a median four times the graph-search budget (HNSW ef). We report both findings with their limits, the registered evaluations that preceded them, and the released models.

1. The index stays; the query encoder changes

In a dense retrieval system the document index is the asset with inertia. Encoding the 23.7 million documents of the BEIR-15 collection with Stella took a measured 39.9 hours on one RTX 3080, and a production index also carries its graph, its quantization, and every downstream consumer of its vectors. The query encoder has no such inertia. It runs once per request, it can be swapped between requests, and nothing stored depends on it, provided its output lands in the document space.

Query-side distillation makes that proviso hold. A student encoder is trained to reproduce the teacher’s query vectors, so it can search the teacher’s document vectors directly. QED, EmbedDistill, and LEAF train transformer students this way [QED, EmbedDistill, LEAF]. pyNIFE fits a static token table to a frozen teacher and proposes switching between static and contextual paths [pyNIFE]. LightRetriever trains a lookup query side jointly with its document tower [LightRetriever]. Static sentence embeddings and Model2Vec supply other table-based representations with their own indexes, and NanoVDR distills a vision-language retriever into a small text encoder [StaticEmb, Model2Vec, NanoVDR]. We claim no priority for the architecture. What this paper adds is what becomes measurable once the student is cheap to build.

We built **Constella** around **Stella**, the `stella_en_400M_v5` embedding model, whose 1024-dimensional

document vectors are fixed throughout [Stella]. **Nano** is a 34.5-million-parameter transformer trained to match Stella’s query vectors. **Zero** is a table of 30,522 learned token vectors; a query is the normalized, count-weighted sum of its tokens’ rows. Stella’s own query encoder remains the third path. All three search the same populated collections without re-encoding or re-indexing, which we verified on live Qdrant collections with no rebuild between encoder batches.

Two findings follow from the cheapness of the students, and the paper is organized around them.

- **Finding A, construction.** Fitting a table student takes four to seven minutes per teacher on an A100, target encoding included, so we could screen 26 teachers under one recipe, then repeat the screen with a second student representation. The teacher’s own retrieval quality was a poor standalone guide to the student’s quality under either recipe. A cheap development screen of the student was a good one.
- **Finding B, serving.** A student’s queries are not the teacher’s queries, and they sit differently against the document graph. Over the unchanged index, table queries need more graph effort to recover their exact neighbors, in all 25 measured teacher spaces on the two larger corpora.

The remaining sections give the setup, the two findings, what the shared index then allows, and the limits. “Registered” marks the project’s pre-registered evaluations, whose methods and decision rules were fixed before any result was observed. “Exploratory” marks analyses added afterwards. The two screens that extend Finding A and Finding B across teachers are exploratory: their analyses were written down before scoring, but they were not part of the registered programme. The registered evaluations that gated the project, including the ones Constella did not pass, are in Appendix A.

2. The family and the setup

Compatibility. Stella produces normalized 1024-dimensional vectors. Zero and Nano are trained against that space and emit vectors of the same width and norm. Zero’s rows are exported as int8 and pooled with square-root count weights, so a repeated token gains influence more slowly than its count. Nano starts from bge-small, a pretrained small English encoder, concatenates the mean-pooled outputs of three of its layers, projects them to 1024 dimensions, and is trained with a squared-error loss against Stella’s query vectors. The prompt, tokenizer, pooling, normalization, and model revision are part of the compatibility contract; switching encoders means selecting an available aligned path, not timed model loading or failover.

Quality and cost. We use **nDCG@10**, which rewards relevant documents near the top of the first ten results, macro-averaged over the 15 BEIR datasets [BEIR]. Exact search scores every document, so these numbers isolate encoder quality from approximate-search error. **Retention** is the ratio of a student’s macro score to Stella’s. Encoding times are medians over 100 real 5-to-12-word queries, batch one, four CPU threads, Apple M5 Pro, three fresh processes, under a registered protocol.

Table 1. Three query paths over identical Stella document vectors.

Query path	Exact nDCG@10	Retention	Encode p50	Assets
Stella	0.5614	100%	31.6 ms	1669.6 MiB
Nano	0.5081	90.5%	2.25 ms	132.3 MiB
Zero	0.4572	81.4%	0.044 ms	90.1 MiB

Nano keeps nine tenths of Stella’s quality at one fourteenth of its encoding time. Zero keeps four fifths at one seven-hundredth, after a 215 ms load and with tokenization, memory, and retrieval still to pay. These are different quality budgets over one index, not equal-quality speedups.

The macro average hides where each budget holds. Figure 1 gives retention per dataset. Zero spans 0.67 on TREC-COVID and FiQA to 0.96 on climate-fever and Quora; Nano is flat between 0.85 and 0.99 except on FEVER, which was in Zero’s training pool and not in Nano’s. In an exploratory analysis of 3,727 queries over six datasets, the gap between Stella and Zero correlates with how much Stella’s own score drops when the query’s words are shuffled (Spearman +0.52 per query). That is the strongest of three correlates we

measured; subwords per word (+0.10) and sensitivity to an equal-size random perturbation (+0.07) are weak. Both the gap and the shuffle drop are computed from Stella’s score, which couples them.

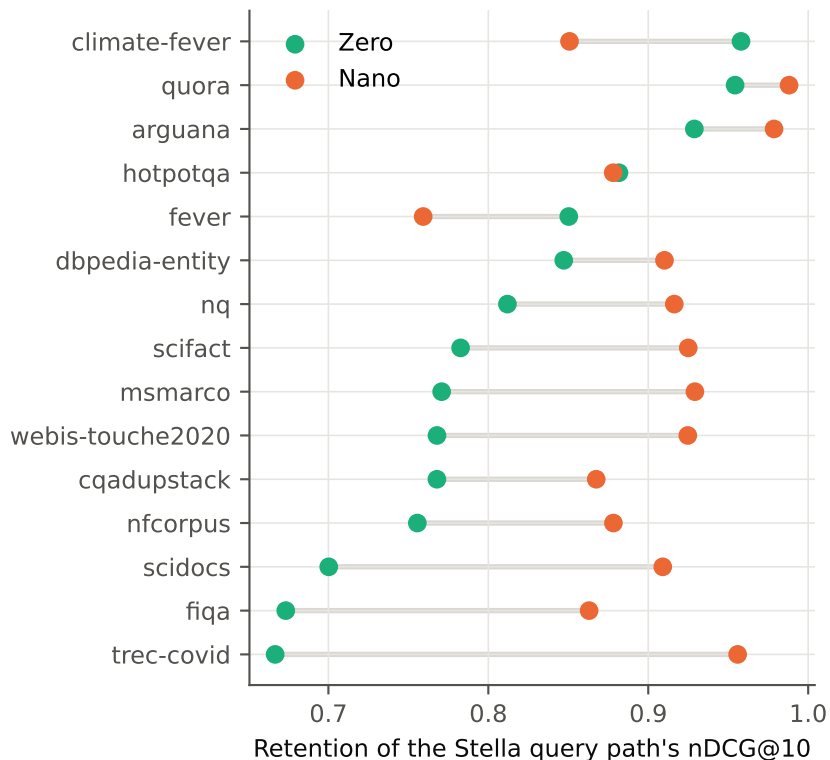


Figure 1: Retention per dataset. Datasets with disclosed Stella training exposure are ArguAna, FiQA, and FEVER; the four originally held-out sets are FEVER, DBpedia-entity, and the CQADupStack android and english forums.

3. Finding A: teacher quality does not rank student quality

An existing index fixes the teacher. Before an index exists, a team may choose which teacher its cheap student will imitate, and the obvious heuristic is to take the strongest one. Early in the project a table screen reversed such a choice, so we tested the heuristic at scale.

The screen. One closed-form recipe fits a token table to each teacher: ridge regression from token counts to the teacher’s query vectors over 337,981 screened queries. The penalty is chosen on two development forums and frozen before any public dataset is scored. Each table searches its own teacher’s document vectors on six BEIR datasets, so this is a teacher-plus-index selection study, not a way to move a student between spaces. Ten teachers were registered; 16 more were added as an exploratory expansion under the same recipe and selection order. All 26 share one WordPiece vocabulary.

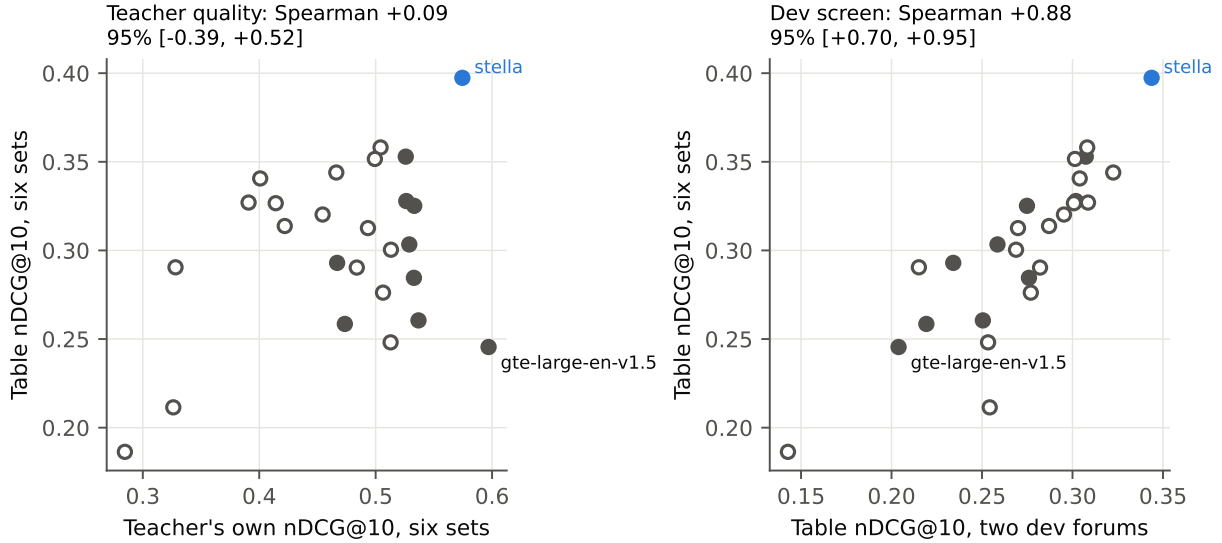


Figure 2: The table recipe. Filled points are the ten registered teachers. Intervals resample checkpoints, not training runs.

The strongest registered teacher, gte-large-en-v1.5, scores 0.5970 on the six datasets and produces the weakest registered table, at 0.2455. Stella scores 0.5745 and produces the best, at 0.3974. Across all 26, teacher quality and table quality have a rank correlation of +0.09 (95% interval -0.39 to $+0.52$), while the table’s two-forum development score and its public score correlate at +0.88 ($+0.70$ to $+0.95$). The development screen is not perfect: on the four datasets without disclosed Stella exposure its agreement drops to +0.68, and the best table varies by dataset.

A second student representation (exploratory, analysis pre-specified before scoring). A referee’s first objection is that this is a property of ridge-fit tables. We therefore repeated the screen with a second student under the same screening setup: a frozen bge-small backbone, the same three-layer feature path Nano uses, with a closed-form ridge head fit to each teacher’s query vectors. Fit list, selection order, and indexes stayed the same. Only the student representation changed.

Rank correlation, 26 teachers	Spearman	95% interval
Head quality vs teacher’s own score	+0.09	-0.36 to $+0.51$
Head quality vs table quality	+0.51	$+0.12$ to $+0.79$
Head development screen vs head public score	+0.84	$+0.61$ to $+0.95$

Under this recipe the strongest registered teacher gives a head of 0.223; the teacher the head’s own development screen selects, bge-base-en-v1.5, gives 0.325. The finding, then: across 26 checkpoints, teacher score alone ranked these students poorly under either tested recipe, the preferred teacher depended on the recipe, and each recipe’s own development screen tracked its public ranking. Screen candidates with the student recipe you intend to use. The one qualifier that belongs beside the finding is width. Both recipes retain less from wider teachers (Spearman -0.73 for the head and -0.76 for the table against teacher dimension). Within one width band, teacher quality does track head quality ($+0.47$, $+0.67$, and $+0.36$ over nine, ten, and seven teachers at 384, 768, and 1024 dimensions). Width covaries with model family and with teacher quality and the bands are small, so this is a hypothesis about what a cheap student pays for a wider space, not a mechanism. Stronger-teacher, weaker-student effects are known in distillation generally [Cho and Hariharan

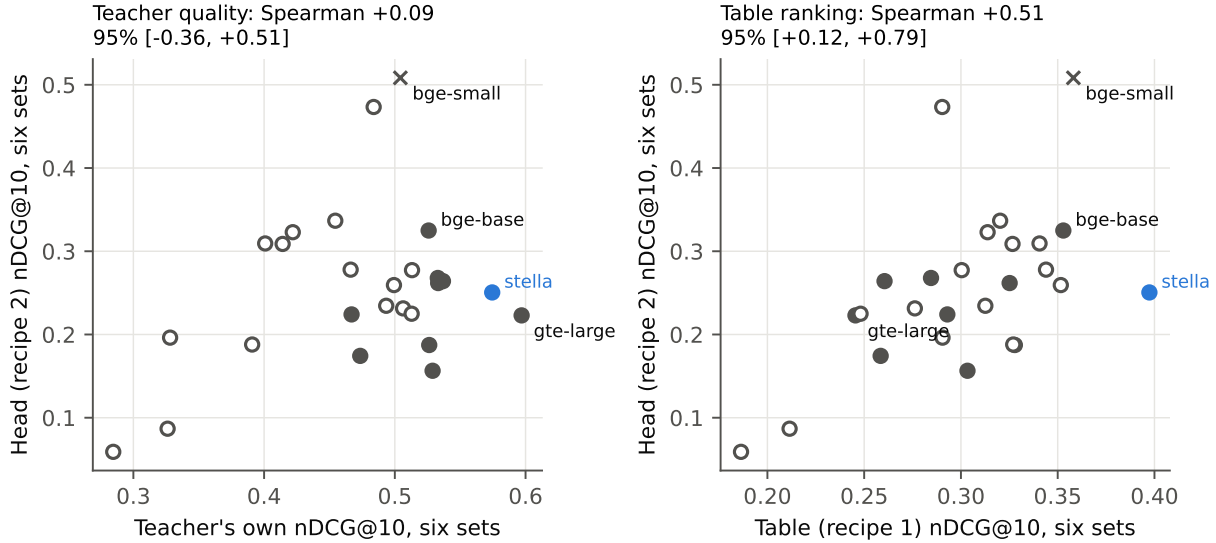


Figure 3: The head recipe. The cross marks bge-small-en-v1.5, which is the head’s own backbone; the correlations change by less than 0.06 without it.

2019, PROD]; our contribution is the consequential reversal, its repetition under a second student representation, and the screen that avoids it. Registered-ten correlations, family sensitivity, and the observation that imitation fidelity does not predict retrieval are in Appendix B.

What the screen costs. Backbone features for the 337,981 fit queries took 43 seconds on one A100, and each teacher’s head grid took 14 seconds; the table recipe took four to seven minutes per teacher including query encoding. These are the fitting components only. Nano’s final optimization ran 57.3 hours on one A100 for 199,999,721 example presentations, about \$95 at the recorded rental rate, excluding target preparation, data generation, recipe search, failed runs, and evaluation. Zero’s retraining was estimated at 20 minutes with prepared targets, plus 8 to 12 hours to encode targets for a new teacher. Cheap fitting is what makes the screen affordable; it is not a complete build budget.

4. Finding B: cheap queries change the search problem

A graph index is built over the document vectors, and a greedy search over it succeeds or fails depending on where the query sits relative to them. Approximate nearest-neighbor search (**ANN**) over a Hierarchical Navigable Small World graph (**HNSW**) explores a bounded set of candidates, controlled by the parameter **ef**: a larger **ef** is more search effort and usually more of the exact neighbors found [HNSW]. Queries that sit off the document distribution are known to make graph search harder, and graph constructions exist for that case [OOD-DiskANN, RoarGraph]. A student’s queries land in the teacher’s space, but not where the teacher’s queries land, which is the hypothesis this section tests.

In Stella’s space. We swept **ef** for all three paths over the same uncompressed Qdrant collections on FiQA (57,638 documents) and a one-million-passage MS MARCO diagnostic that keeps every judged positive. The protocol was registered, with exact parity checked against a NumPy reference. Search time at a fixed **ef** is similar for all three paths in these measurements (1.60, 1.53, and 1.53 ms at **ef**=128 on the million-passage collection). What differs is how much **ef** each path needs. To stay within 1% of its own exact score, Stella needs **ef**=128, Nano 256, and Zero 512, with search medians of 1.53, 2.18, and 3.58 ms. At **ef**=16 the losses are 4.4%, 7.0%, and 16.3%; with binary quantization and no oversampling they are 6.1%, 9.2%, and 19.8%. Figure 4 shows the full curves.

On one shared binary collection, an exploratory restriction of the registered sweep, the fastest setting within 1% of each path’s own exact score is: Zero 0.6116 at 1.11 ms (**ef**=512, 2x oversampling); Nano 0.6893 at

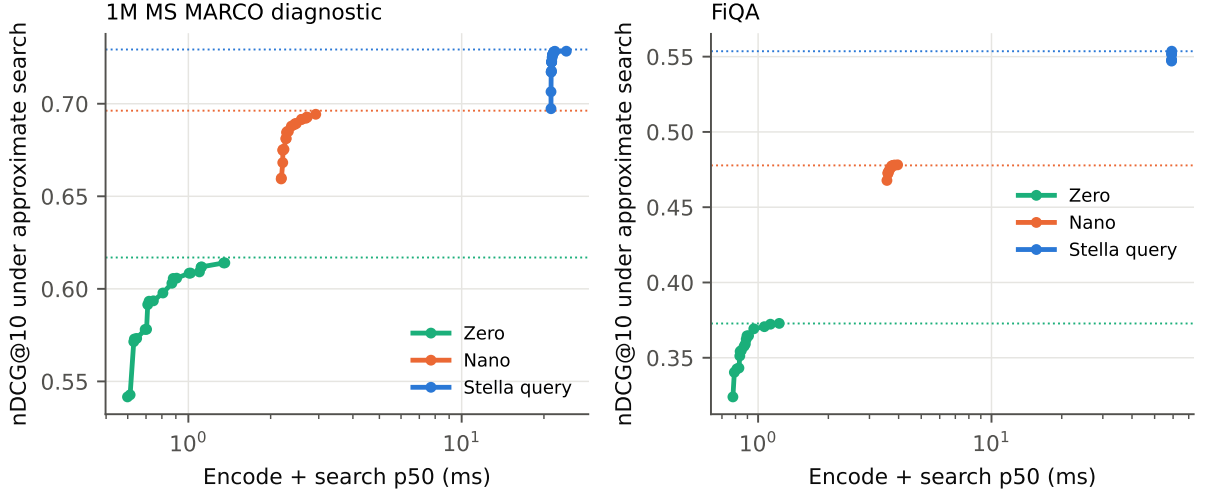


Figure 4: Quality under approximate search against encoding plus search time, registered sweep. Dotted lines are each path’s exact score. Curves may use different compression settings; times sum separately measured per-query phases and exclude loading and dispatch.

2.48 ms ($ef=256$, 4x); Stella 0.7226 at 21.3 ms ($ef=128$, 1x). The encoding gap between Zero and Nano is 63-fold on these queries (51-fold on Table 1’s sample); after search it is 2.2-fold, at different quality levels. Most of that compression is arithmetic: search adds about a millisecond to both paths, which is 40 times Zero’s encoding time and half of Nano’s. The part specific to the cheap encoder is the extra ef .

Zero’s queries also sit differently in the space: median top-1 cosine to the documents is 0.60 for Zero against 0.74 for Nano and 0.78 for Stella, and the gap between the first and tenth scores is flattest for Zero. That is consistent with harder graph navigation, and the next experiment tests whether it predicts anything.

Across 25 teacher spaces (exploratory, analysis pre-specified before scoring). Is the extra effort a property of Stella, or of table-based query substitution? For every teacher in the screen we built uncompressed HNSW indexes over its own document vectors on FiQA, SCIDOCS (25,657 documents), and TREC-COVID (171,332 documents), with $m=16$, $ef_construct=100$, and two builds with different insertion orders. We then swept ef from 16 to 512 for the teacher’s queries, its table’s queries, and its head’s queries. Each path is measured against its own exact top-10, and exact parity was checked on every collection. One teacher, tas-b, was excluded because its exact top-10 is undefined at float32 precision on these corpora. The analysis was fixed before scoring. The teacher’s reference is $ef=64$. The multiplier is the smallest ef at which the table reaches the teacher’s reference value, divided by 64, averaged over builds, and censored when a build never reaches it by 512.

The primary, pre-specified measure is the relative-nDCG-loss multiplier. On FiQA every one of the 25 spaces needs more effort than the teacher: the median over the 21 spaces that reach the teacher’s loss by $ef=512$ is four times, and four spaces are censored above eight. On SCIDOCS 17 spaces need more effort and eight do not (median 1.25 over 17 reached, eight censored). On TREC-COVID the result is mixed: 12 spaces above one and 13 at or below, with a median of 0.25 over 17 reached and eight censored. Its teachers lose only 0.2% of nDCG at $ef=64$, so a threshold on relative loss over 50 queries is sensitive to small differences. That is a plausible explanation, not a reason to set the result aside. The secondary, pre-specified measure is exact-neighbor recovery, and it reads consistently. Table queries recovered fewer of their own exact neighbors at $ef=64$ in all 25 spaces on FiQA (mean -2.9 percentage points, worst -11.8), all 25 on TREC-COVID (-3.9 , worst -18.3), and 17 of 25 on SCIDOCS (-0.8). Matching the teacher’s recovery took a median four times the ef over reached spaces on FiQA and TREC-COVID, with the tenth and ninetieth percentiles at two and six, and a median two on SCIDOCS, with one space censored per workload. The head recipe shows the same recovery deficit (24 and 23 of 25 spaces). Corpus size is a plausible reason the effect shrinks on

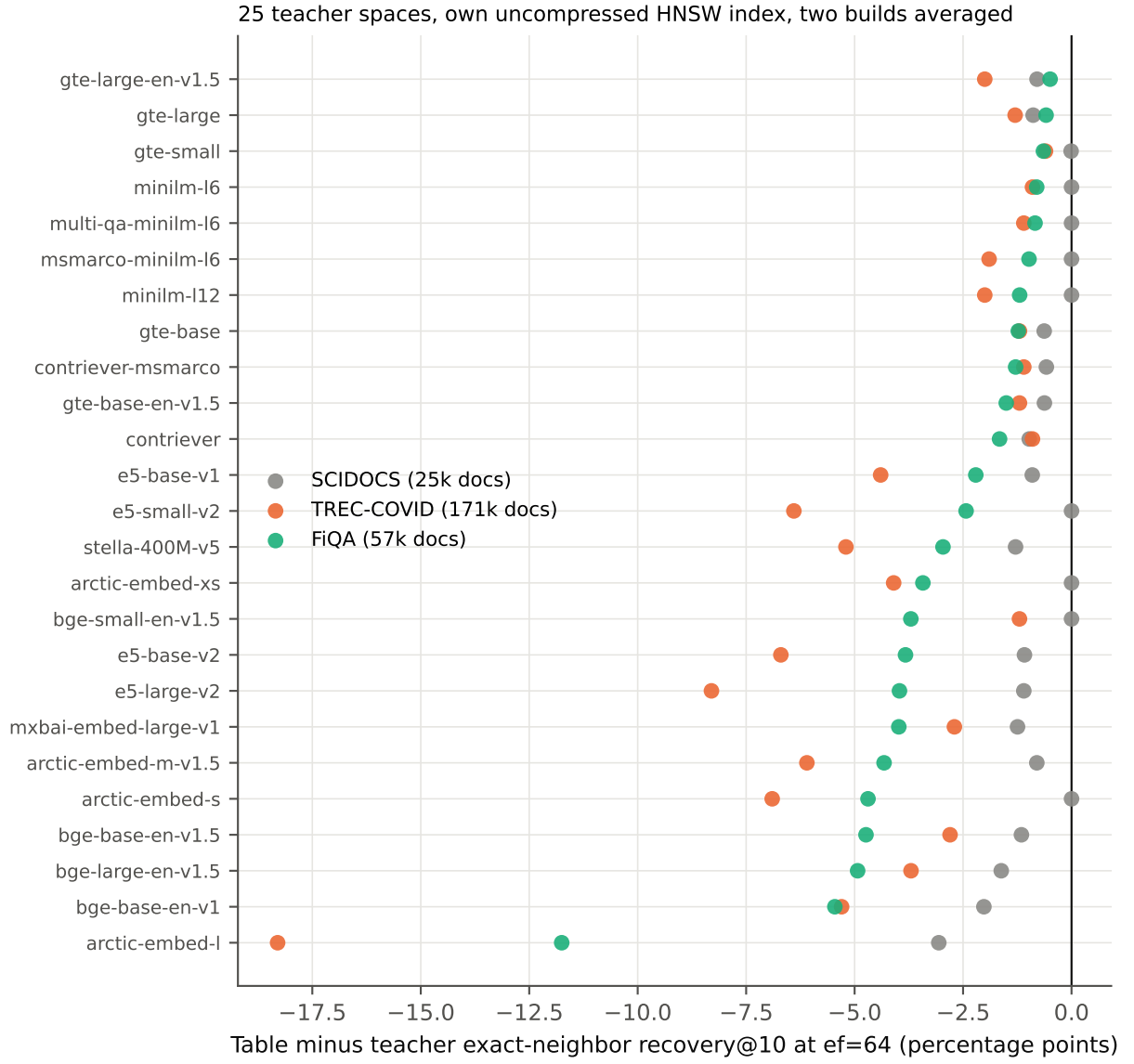


Figure 5: The table recovers fewer of its own exact neighbors than the teacher does at the same ef , in every space on FiQA and TREC-COVID. Two builds averaged.

SCIDOCS, not an identified cause, since three datasets confound size with workload.

The two geometry summaries we measured, top-1 cosine and top-1-to-top-10 margin, showed no consistent association with the recovery penalty across workloads (Spearman 0.00 on FiQA, +0.23 on SCIDOCS, -0.34 on TREC-COVID, intervals spanning zero). The penalty recurs; its size is not yet predictable from these summaries. Four times the **ef** is not four times the latency, and nothing here measures a production request.

Query precision is a separate budget (exploratory). Binary document codes keep coordinate signs, and the tested engine’s default binary query encoding keeps only the query’s signs as well [Qdrant 1.19.1]. With document codes fixed and no graph, scoring with full query magnitudes instead of signs raises Zero’s coverage of its exact top-10 from 82.2% to 93.0% on FiQA at a 40-candidate budget. Every path gains, in line with query-aware decoding of binary sketches [QA-Cos]. The engine’s magnitude-preserving query setting was not tested here; this is a coverage result, not a latency or relevance result (Appendix C).

5. What the shared index then allows

Because the three paths query one populated collection, an application can choose its budget per workload or per device without touching the index. Two further options were measured (Appendix D). Fusing Zero with BM25 recovers more quality than any table-side tuning did, raising its BEIR-15 macro from 0.4572 to 0.4933, at the cost of a sparse index and a second retrieval, with a sign that varies by workload [DBSF]. Per-query routing between Zero and Nano has real headroom: an oracle gains 0.031 over always-Nano on 12 datasets. The cheap selectors we tested recover little of it, and a vector blend selected on development data lost quality outside it, which fits how hard retrieval-strategy selection remains [Arabzadeh et al., QPP]. Compatible vectors permit these policies; they do not justify them by themselves.

6. Limits and implications

Finding A rests on two closed-form student recipes over 26 related checkpoints. A frozen head is a floor for a trained student, the screen does not select the trained Zero or Nano, and checkpoint intervals do not cover training variance. Finding B rests on one engine, one HNSW configuration, and closed-form tables outside Stella’s space, where the trained Zero was not measured. Its three workloads confound corpus size with domain, and its timing ran on a shared machine, so it is not a latency claim. Zero and Nano have different training pools, so their comparison does not isolate architecture, and Stella discloses training exposure on ArguAna, FiQA, and FEVER. Query intervals condition on the trained models. The geometry null is exploratory.

What a search engineer can take from this: screen the student you intend to build. Expect a cheap student to need a larger search budget over the same graph, and measure encoding plus search together. Treat query precision as its own knob. The next experiments with the clearest value are a native, cost-qualified query-precision comparison and a trained student in a second teacher space.

Artifact availability

The companion repository holds inference examples, build configurations, methods with their dated amendments, immutable result receipts, per-dataset tables, and the full research history including negative results. The released artifacts are Constella Zero, Constella Nano, and the Stella document encoder on the Hugging Face Hub. All released paths run in fp32 through the published loader; a candidate fp16 export passed CPU parity and failed on CUDA (minimum cosine 0.662), which is why qualification runs through the actual loader and device. Two inputs are identified by hash but not yet archived in the repository: the cleaned 337,981-query fit list and the sampled one-million-passage identifier list.

Appendix A. Registered evaluations

Serving configuration. Stella is pinned to revision **ffeb2b7e**, with its **s2p_query** prompt, fp32 encoding, normalized output, fp16 stored document vectors, and cosine similarity; maximum sequence length 512. Nano uses bge-small layers 12, 8, and 4 concatenated to 1152 dimensions before projection; batches mix 75% queries

and 25% documents. Zero’s training pool includes FEVER-train and HotpotQA; Nano excludes FEVER. MS MARCO is licensed for non-commercial use, so it was validation-only in our distillation: it supplied no training targets, negatives, or generation seeds, although Nano’s pretrained base has prior exposure to it.

Partitions. The six-set suite is NFCorpus, SCIDOCS, SciFact, TREC-COVID, FiQA, and ArguAna. “Clean-four” means the first four, which have no disclosed Stella overlap; it is not a causal contamination estimate. Four datasets were held out and evaluated once: FEVER, DBpedia-entity, and the CQADupStack android and english forums. Two other forums, physics and programmers, select table penalties and routing settings and were reused during model construction.

Reference systems. bge-small-en-v1.5 and LEAF were the project’s selected targets, not a state-of-the-art frontier. They use their own document representations. On BEIR-15 they score 0.5171 and 0.5402 against Nano’s 0.5081, at 2.57 and 1.39 ms median encoding; BM25 scores 0.4006. Nano is below both on the broad average even though it passed some of the registered contrasts below.

Nano’s registered contrasts used a fixed sequence with one-sided 2.5% lower bounds. “Established” means the bound cleared zero; “unresolved” means it did not; “descriptive” rows were registered as reports, not tests. Intervals resample queries with the models fixed.

Contrast	Partition	nDCG@10 difference	Outcome
Nano – bge-small	clean-four	+0.0176; lower bound +0.0037	established
Nano – bge-small	all six	+0.0274; lower bound +0.0173	established
Nano – LEAF	all six	+0.0162; lower bound +0.0065	established
Nano – LEAF	clean-four	−0.0011; lower bound −0.0145	unresolved
Nano – bge-small	held-out NDO-3, dataset weighted	+0.0032 [−0.0069, +0.0134]	descriptive
Nano – bge-small	held-out NDO-3, query pooled	−0.0121 [−0.0219, −0.0024]	descriptive
Nano – LEAF	held-out NDO-3	−0.0389 [−0.0488, −0.0290]	descriptive

NDO-3 is the registered held-out aggregate: DBpedia-entity weighted one half, the android and english forums one quarter each, FEVER excluded as the double-exposure sensitivity row.

Zero’s registered contrasts used a Holm correction across the family. Zero missed its dense bar and did not confirmatorily beat BM25.

Contrast, all six	nDCG@10 difference	Raw 95% interval	Outcome
Zero – LightRetriever dense	−0.0243	[−0.0405, −0.0086]	below the 0.4583 bar
Zero – BM25	+0.0165	[+0.0017, +0.0311]	unresolved; p=.0149 fails Holm .0083
Zero + BM25 (convex) – OpenSearch	+0.0043	[−0.0063, +0.0151]	unresolved

The registered clean-four sensitivity is descriptive: −0.0443 [−0.0675, −0.0212], −0.0311 [−0.0517, −0.0109], and −0.0107 [−0.0262, +0.0043]. Unresolved superiority does not establish equivalence. The confirmatory fusion operator was convex score fusion with development-fitted weight 0.8 at depth 1000; the later DBSF-at-100 recommendation is a deployability choice, not a new confirmation. On the development components DBSF minus convex is +0.0035 at 10 candidates, −0.0020 at 50, −0.0063 at 100, and −0.0146 at 1000.

The held-out four, copied from their one-shot published aggregate:

System	FEVER	DBpedia	CQADup android	CQADup english
Nano	0.6231	0.4190	0.4774	0.4298
Zero	0.6978	0.3900	0.4431	0.3690
Stella	0.8207	0.4603	0.5275	0.5183
bge-small	0.8662	0.4003	0.4761	0.4558
LEAF	0.8671	0.4520	0.5260	0.4707
BM25	0.5036	0.3045	0.3952	0.3481
Zero + BM25	0.7559	0.4030	0.4629	0.4016
Nano + BM25	0.6879	0.4296	0.4750	0.4323

Appendix B. Construction details

Table recipe. Ridge regression from WordPiece token counts to the teacher’s query vectors, initialized from the teacher’s own token embeddings, solved by block conjugate gradient with a convergence gate; penalty grid 1e-4 to 1e-1 with one decade of extension at an edge, chosen on the two development forums and frozen before six-set scoring. The fit list was cleaned after an earlier version had 1.31% held-out-query overlap. One checkpoint failed the convergence gate and is excluded. Registered checkpoint exposure is documented; the exploratory expansion was not individually audited.

Head recipe. Frozen bge-small at the Nano build’s pinned revision; masked-mean-pooled hidden states of layers 12, 8, and 4 concatenated with a bias column; dense ridge with penalty scaled by the feature Gram trace, same grid, same selection order, same freeze; output normalized. bge-small-en-v1.5 is also a teacher and shares the backbone, and gte-small is nearly its linear image (retention 1.008 and 0.978); correlations without the backbone are +0.09 against the teacher and +0.46 against the table.

Correlation rosters. Registered ten: Spearman head-versus-teacher +0.18 (−0.56 to +0.67), head-versus-table +0.24 (−0.49 to +0.74), head development-versus-public +0.71 (+0.12 to +0.96). Leave-one-family-out values range from +0.44 to +0.60 for head-versus-table and −0.12 to +0.27 for head-versus-teacher over the 26.

Fidelity is not retrieval. Tables fitted to two e5 checkpoints match their teachers’ query vectors at mean cosine 0.90 and 0.89 against 0.78 for Stella’s table, yet retrieve worse; three earlier training screens reduced their losses without improving ranking. Three vector-error diagnostics did not rank table quality either.

Costs. Table fitting four to seven minutes per registered teacher on an A100, inferred from output intervals, including query encoding except cached Stella targets. Head features 43 seconds for the fit list; dev grid 14 seconds per teacher; six-set scoring about one minute per teacher. These exclude downloads, data assembly, and index construction.

Appendix C. Search and precision controls

Stella-space sweep. Qdrant 1.19.1, HNSW m=16, ef_construct=100, indexing threshold 1 KB so no segment is served by a plain scan, ef in 16 to 512, oversampling 1, 2, and 4 under compression, exact parity against NumPy on 200 queries per path. Timing is warmed and sequential with encoding and search measured in separate phases.

Cross-space sweep. The same engine and graph parameters, uncompressed, two builds per space and workload differing in a seeded insertion order, 16 to 512 ef, limit 11 with the self-hit drop, exact parity checked tie-aware on 200 queries per path. Recovery@10 counts a returned document when its exact score is at or above the exact tenth neighbor’s score within 1e-4, because some tables produce exact-score ties at rank ten and two exact scorers legitimately break them differently; the set-overlap version is kept in every row. Four amendments were recorded during the run, before any space’s rows were analysed: SCIDOCS added as a third workload because TREC-COVID has 50 queries; the tie-aware measure; a 0.995 sanity gate on parity with all values reported; and the exclusion rule that removed tas-b (SCIDOCS head parity 0.994). Stella’s teacher rows agree with the Stella-space sweep at every ef. Censoring: on FiQA four spaces never

reached the teacher’s relative loss by **ef=512** in at least one build; on SCIDOCS eight; on TREC-COVID eight.

Native binary pilot and replication. On one fixed binary collection per workload with 192 deterministically selected queries, binary scoring with rescoring at 1x oversampling loses 11.7, 5.1, and 2.3 percentage points of exact-neighbor recovery for Zero, Nano, and Stella on FiQA at **ef=64**; the million-passage replication loses 3.9, 3.1, and 2.2, and the extra Zero-versus-Nano interval includes zero. Native rescoring also changes cross-segment candidate merging.

Graph-free precision control. Same queries, vectors, and targets; all document sign codes scored exhaustively at global candidate budgets 10, 40, and 100; expected coverage averages uniform inclusion at tied boundaries.

Workload	Path	Sign query	Full query	Gain, points
FiQA	Zero	0.8217	0.9297	+10.80
FiQA	Nano	0.9137	0.9771	+6.33
FiQA	Stella	0.9638	0.9922	+2.84
1M diagnostic	Zero	0.9010	0.9677	+6.67
1M diagnostic	Nano	0.9484	0.9833	+3.49
1M diagnostic	Stella	0.9685	0.9932	+2.47

The paired extra Zero gains over Nano are 4.46 points on FiQA (95% query interval 2.53 to 6.43) and 3.17 on the diagnostic (1.58 to 4.75). Zero starts with more misses and does not repair a larger fraction of them. Full query values are not native scalar8 encoding, and global candidate counts are not per-segment oversampling.

Appendix D. Fusion and routing over the shared index

Lexical fusion. Fusing Zero with BM25 raises its BEIR-15 macro score from 0.4572 to 0.4933, 87.9% of Stella; Nano plus BM25 reaches 0.5110. The sign varies by workload: Zero on HotpotQA rises from 0.6127 to 0.7015, Nano on MS MARCO falls from 0.4063 to 0.3699. On the million-passage diagnostic a dense-plus-sparse collection takes 4.71 ms per query including Zero encoding against 1.11 ms for the dense binary example, in different configurations, not an isolated fusion overhead. The scores use distribution-based score fusion over 100 candidates per branch with a bm25s lexical branch; other lexical implementations need their own comparison because their score distributions enter the normalization. The operator ordering changes with candidate depth (Appendix A).

Routing and blending (registered). On 12 datasets, an oracle choosing Zero or Nano per query with relevance labels scores 0.5473 against 0.5161 for always-Nano, and matches always-Nano by sending the most beneficial 15% of queries to Nano. Query length, subwords per word, and the pooled vector norm are weak selectors (0.002 to 0.005 over random routing at matched Nano share); Zero’s post-search score margin recovers about a quarter of the oracle gain over random routing on six datasets and costs a second search for escalated queries. A development-selected 30% Zero contribution blended into Nano’s vector lowers Nano’s six-set score by 0.0051 (95% paired interval -0.0092 to -0.0011).

References

- [QED] Yuxuan Wang and Hong Lyu. Query Encoder Distillation via Embedding Alignment is a Strong Baseline Method to Boost Dense Retriever Online Efficiency. SustainNLP Workshop at ACL, 2023. [arXiv:2306.11550](#).
- [EmbedDistill] Seungyeon Kim, Ankit Singh Rawat, Manzil Zaheer, et al. EmbedDistill: A Geometric Knowledge Distillation for Information Retrieval. 2023. [arXiv:2301.12005](#).
- [LEAF] Robin Vujanic and Thomas Rueckstiess. LEAF: Knowledge Distillation of Text Embedding Models with Teacher-Aligned Representations. 2025, revised 2026. [arXiv:2509.12539](#).
- [NanoVDR] Zhuchenyang Liu, Yao Zhang, and Yu Xiao. NanoVDR: Distilling a 2B Vision-Language Retriever into a 70M Text-Only Encoder for Visual Document Retrieval. 2026. [arXiv:2603.12824](#).

- [LightRetriever] Guangyuan Ma, Yongliang Ma, Xuanrui Gou, Zhenpeng Su, Ming Zhou, and Songlin Hu. LightRetriever: A LLM-based Text Retrieval Architecture with Extremely Faster Query Inference. ICLR, 2026. [arXiv:2505.12260](#).
- [pyNIFE] Stephan Tulkens. pyNIFE: Nearly Inference Free Embeddings in Python. Software, 2025. [Repository](#).
- [Model2Vec] MinishLab. Model2Vec: Fast State-of-the-Art Static Embeddings. Software. [Repository](#).
- [StaticEmb] Tom Aarsen. Train 400x faster Static Embedding Models with Sentence Transformers. Hugging Face, 15 January 2025. [Blog post](#).
- [Cho and Hariharan 2019] Jang Hyun Cho and Bharath Hariharan. On the Efficacy of Knowledge Distillation. ICCV, 2019. [Proceedings](#).
- [PROD] Zhenghao Lin, Yeyun Gong, Xiao Liu, et al. PROD: Progressive Distillation for Dense Retrieval. WWW, 2023. [arXiv:2209.13335](#).
- [HNSW] Yury A. Malkov and Dmitry A. Yashunin. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. IEEE TPAMI 42(4), 2020. [arXiv:1603.09320](#).
- [OOD-DiskANN] Shikhar Jaiswal, Ravishankar Krishnaswamy, Ankit Garg, Harsha Vardhan Simhadri, and Sheshansh Agrawal. OOD-DiskANN: Efficient and Scalable Graph ANNS for Out-of-Distribution Queries. 2022. [arXiv:2211.12850](#).
- [RoarGraph] Meng Chen, Kai Zhang, Zhenying He, Yinan Jing, and X. Sean Wang. RoarGraph: A Projected Bipartite Graph for Efficient Cross-Modal Approximate Nearest Neighbor Search. PVLDB 17(11): 2735-2749, 2024. [Paper](#).
- [QA-Cos] Daehun Nyang. Beyond Hamming: Query-Aware Decoding of Binary Cosine Sketches. ICML 2026, PMLR 306: 94126-94143. [Proceedings](#).
- [Arabzadeh et al.] Negar Arabzadeh, Xinyi Yan, and Charles L. A. Clarke. Predicting Efficiency/Effectiveness Trade-offs for Dense vs. Sparse Retrieval Strategy Selection. CIKM, 2021. [arXiv:2109.10739](#).
- [QPP] Guglielmo Faggioli, Thibault Formal, Stefano Marchesin, Stéphane Clinchant, Nicola Ferro, and Benjamin Piwowarski. Query Performance Prediction for Neural IR: Are We There Yet? 2023. [arXiv:2302.09947](#).
- [DBSF] Qdrant contributors. Hybrid queries: distribution-based score fusion. Qdrant documentation. [Page](#).
- [Qdrant 1.19.1] Qdrant contributors. Binary query encoding and vector-index search implementation, v1.19.1. [Tagged source](#).
- [Stella] Stella-en-400M-v5. Model card for NovaSearch/stella_en_400M_v5. [Model page](#).
- [BEIR] BEIR contributors. BEIR retrieval benchmark, datasets, and evaluation software. [Repository](#).